

LAB ASSIGNMENT-8.3

P.Chithra

2303A52137

Batch-41

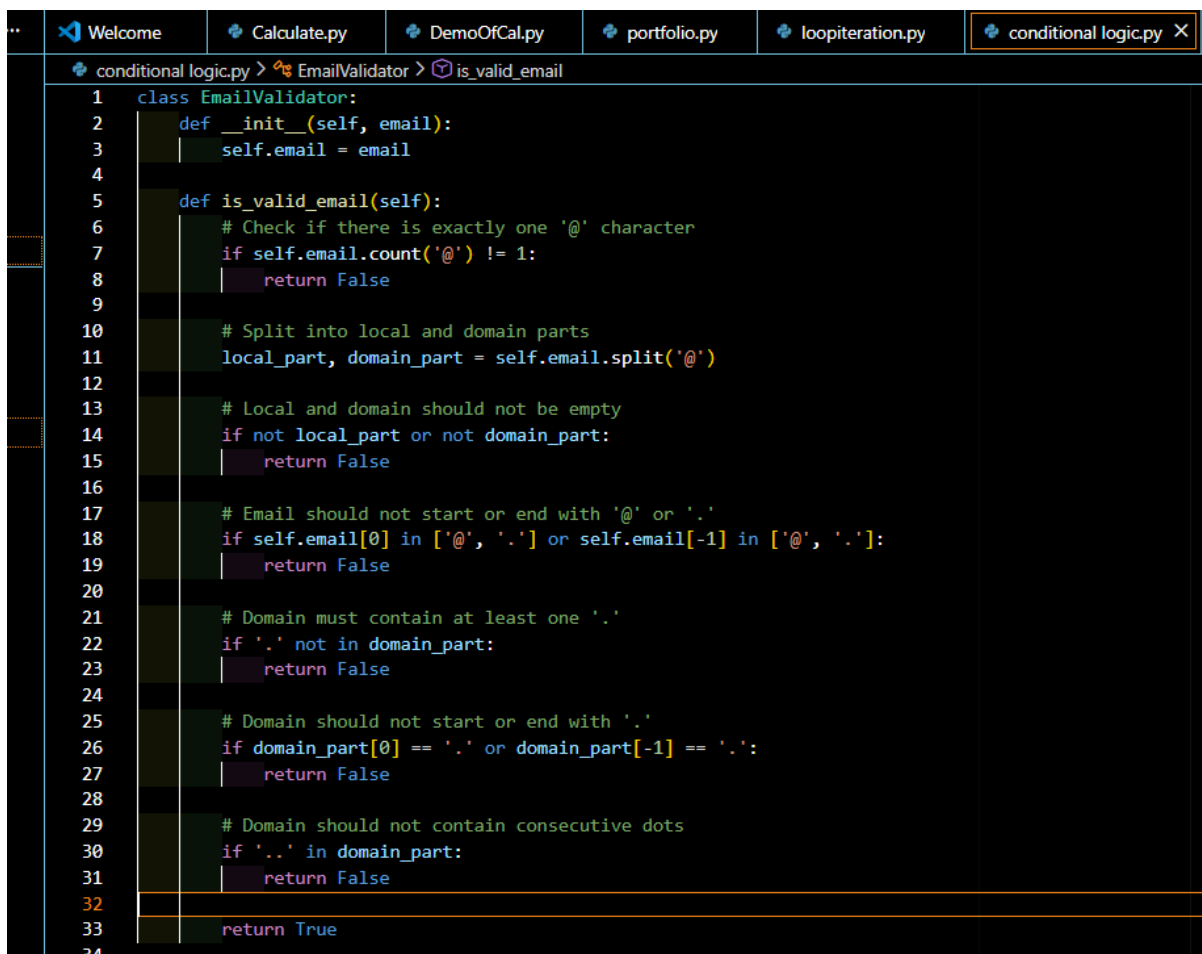
Question 1:

Task 1: Email Validation using TDD

Prompt:

Generate comprehensive test cases for email input validation. The email must contain exactly one @ symbol and at least one . character. It must not start or end with special characters. Include all possible valid and invalid scenarios, covering edge cases as well.

Code:



```
.. Welcome Calculate.py DemoOfCal.py portfolio.py loopiteration.py conditional logic.py X
conditional logic.py > EmailValidator > is_valid_email
1 class EmailValidator:
2     def __init__(self, email):
3         self.email = email
4
5     def is_valid_email(self):
6         # Check if there is exactly one '@' character
7         if self.email.count('@') != 1:
8             return False
9
10        # Split into local and domain parts
11        local_part, domain_part = self.email.split('@')
12
13        # Local and domain should not be empty
14        if not local_part or not domain_part:
15            return False
16
17        # Email should not start or end with '@' or '.'
18        if self.email[0] in ['@', '.'] or self.email[-1] in ['@', '.']:
19            return False
20
21        # Domain must contain at least one '.'
22        if '.' not in domain_part:
23            return False
24
25        # Domain should not start or end with '.'
26        if domain_part[0] == '.' or domain_part[-1] == '.':
27            return False
28
29        # Domain should not contain consecutive dots
30        if '..' in domain_part:
31            return False
32
33        return True
34
```

```
34
35
36 # ----- Test Cases -----
37
38 test_cases = [
39     "test@example.com",
40     "user.name@domain.co",
41     "user_name@domain.com",
42     "username@sub.domain.com",
43     "username@domain.co.in",
44     "user-name@domain.com",
45     "user.name+tag+sorting@example.com",
46     "user.name@example.co.uk",
47     "user.name@domain",
48     "user.name@.com",
49     "@example.com",
50     "username@domain..com",
51     "username@domain.c",
52     "username@@domain.com"
53 ]
54
55
56 def run_tests():
57     for email in test_cases:
58         validator = EmailValidator(email)
59         result = validator.is_valid_email()
60         print(f"Email: {email}, Valid: {result}")
61
62
63 if __name__ == "__main__":
64     run_tests()
65
```

Output:

```
Email: username@@domain.com, Valid: False
D:\Downloads\Python_FullCourse>C:/Users/Chithra/AppData/Local/Programs/Python/Python313/python.exe "d:/Downloads/Python_FullCourse/conditional logic.py"
Email: test@example.com, Valid: True
Email: user.name@domain.co, Valid: True
Email: user.name@domain.com, Valid: True
Email: username@sub.domain.com, Valid: True
Email: username@domain.co.in, Valid: True
Email: user-name@domain.com, Valid: True
Email: user.name+tag+sorting@example.com, Valid: True
Email: user.name@example.co.uk, Valid: True
Email: user.name@domain, Valid: False
Email: user.name@.com, Valid: False
Email: @example.com, Valid: False
Email: username@domain..com, Valid: False
Email: username@domain.c, Valid: True
Email: username@domain.com, Valid: False
D:\Downloads\Python_FullCourse>
```

JUSTIFICATION:

The prompt clearly asks to validate email addresses based on specific rules like having exactly one @, at least one . and not starting or ending with special

characters. The code follows these rules by first checking the count of @, then ensuring a dot is present. It uses a regular expression to make sure the overall format of the email is correct. The function returns True if all conditions are satisfied, otherwise False. The output displays each email along with whether it is valid or not. This makes it easy to test multiple cases and verify the email validation logic clearly.

QUESTION 2

Task 2: Grade Assignment using Loops

Prompt:

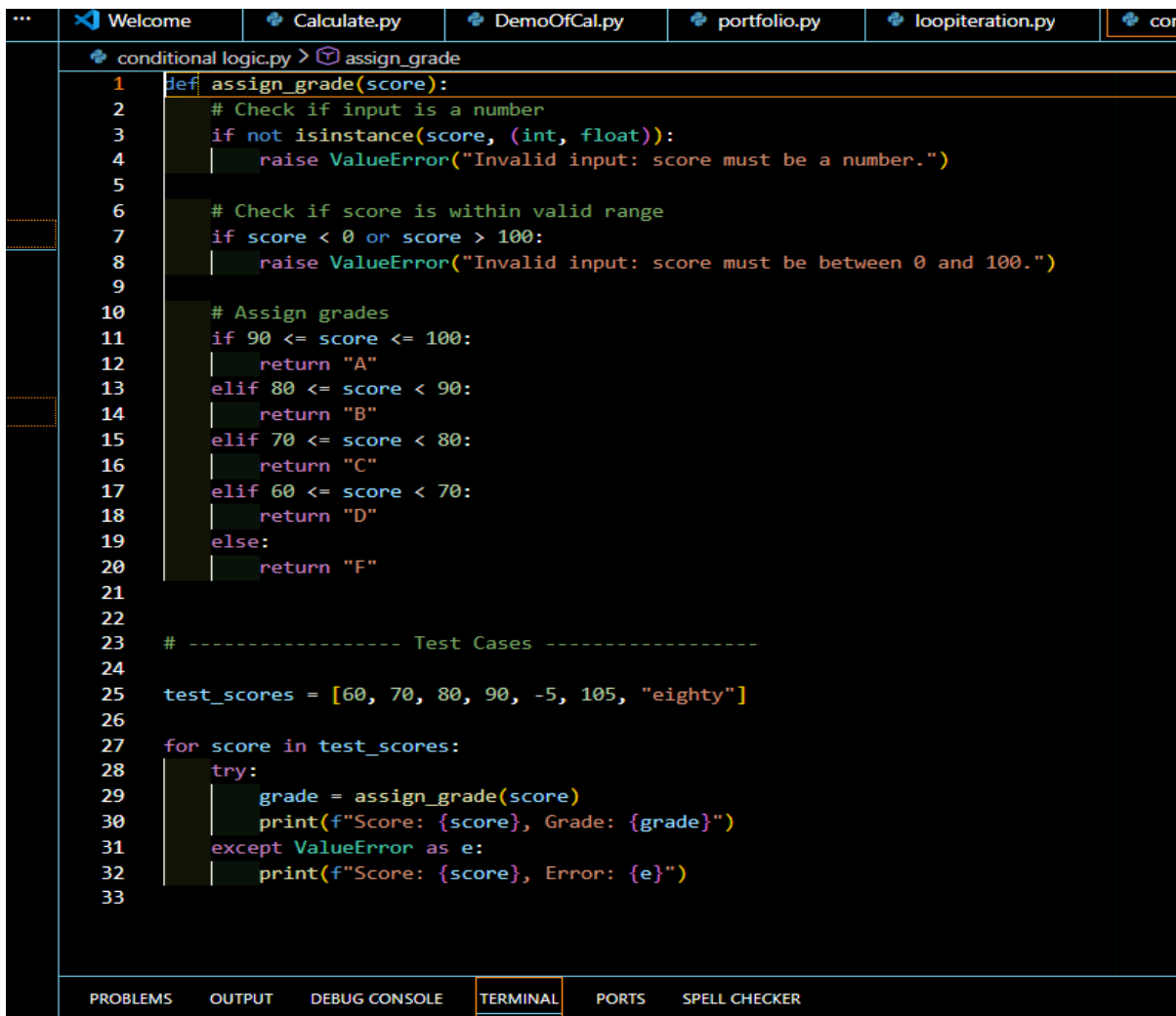
Write a Python function called `assign_grade(score)` that gives a grade based on marks.

If the score is 90–100 return "A", 80–89 return "B", 70–79 return "C", 60–69 return "D", and below 60 return "F".

The function should also check for wrong inputs like negative numbers, numbers greater than 100, or text values.

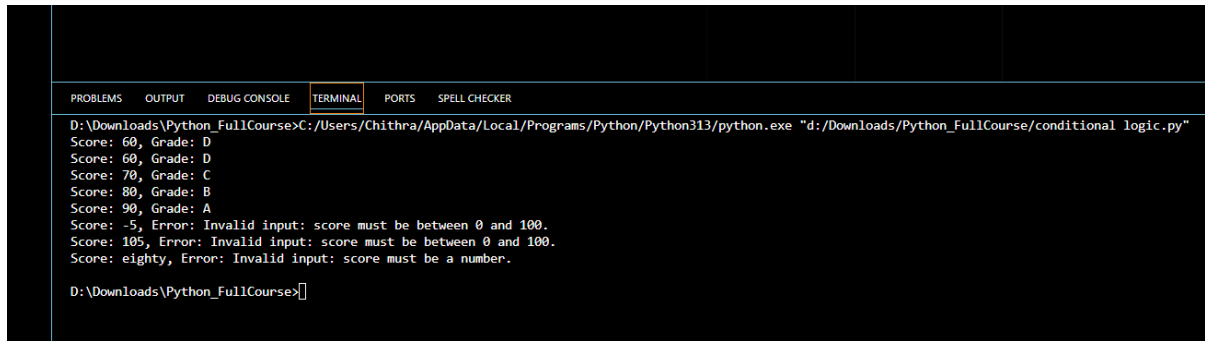
Test the function using these values: 60, 70, 80, 90, -5, 105, and "eighty".

Code:



```
1 def assign_grade(score):
2     # Check if input is a number
3     if not isinstance(score, (int, float)):
4         raise ValueError("Invalid input: score must be a number.")
5
6     # Check if score is within valid range
7     if score < 0 or score > 100:
8         raise ValueError("Invalid input: score must be between 0 and 100.")
9
10    # Assign grades
11    if 90 <= score <= 100:
12        return "A"
13    elif 80 <= score < 90:
14        return "B"
15    elif 70 <= score < 80:
16        return "C"
17    elif 60 <= score < 70:
18        return "D"
19    else:
20        return "F"
21
22
23 # ----- Test Cases -----
24
25 test_scores = [60, 70, 80, 90, -5, 105, "eighty"]
26
27 for score in test_scores:
28     try:
29         grade = assign_grade(score)
30         print(f"Score: {score}, Grade: {grade}")
31     except ValueError as e:
32         print(f"Score: {score}, Error: {e}")
33
```

Output:



```
D:\Downloads\Python_FullCourse>C:/Users/Chithra/AppData/Local/Programs/Python/Python313/python.exe "d:/Downloads/Python_FullCourse/conditional logic.py"
Score: 60, Grade: D
Score: 60, Grade: D
Score: 70, Grade: C
Score: 80, Grade: B
Score: 90, Grade: A
Score: -5, Error: Invalid input: score must be between 0 and 100.
Score: 105, Error: Invalid input: score must be between 0 and 100.
Score: eighty, Error: Invalid input: score must be a number.

D:\Downloads\Python_FullCourse>
```

JUSTIFICATION:

The function checks whether the input is a valid number before assigning a grade. It ensures the score is between 0 and 100 to handle boundary values properly. If the input is negative, greater than 100, or not a number, it returns a clear error message. The grading conditions are written in order from highest to lowest to make the logic simple and readable. The output clearly shows either the grade or the appropriate error message, making the program easy to understand.

QUESTION 3

Task 3: Sentence Palindrome Checker

Prompt:

Write a Python program to check if a sentence reads the same forward and backward. Do not consider capital letters, spaces, or punctuation marks. Test the program with some sentences that are palindromes and some that are not. For example, "A man a plan a canal Panama" should give True.

Code:

conditional logic.py > ...

```
1  import string
2
3  def is_palindrome(sentence):
4      # Remove spaces and punctuation, and convert to lowercase
5      cleaned_sentence = ''.join(
6          char for char in sentence if char.isalnum()
7      ).lower()
8
9      # Check if cleaned sentence is equal to its reverse
10     return cleaned_sentence == cleaned_sentence[::-1]
11
12
13     # ----- Test Cases -----
14
15     test_sentences = [
16         "A man, a plan, a canal Panama",
17         "No 'x' in Nixon",
18         "Was it a car or a cat I saw?",
19         "Not a palindrome",
20         "12321",
21         "12345"
22     ]
23
24     for sentence in test_sentences:
25         result = is_palindrome(sentence)
26         print(f'{sentence} → Is palindrome: {result}')
27
```

Output:

```
Score: eighty, Error: invalid input: score must be a number.

D:\Downloads\Python_FullCourse>C:/Users/Chithra/AppData/Local/Programs/Python/Python313/python.exe "d:/Downloads/Python_FullCourse/conditional logic.py"
"A man, a plan, a canal Panama" → Is palindrome: True
"No 'x' in Nixon" → Is palindrome: True
"Was it a car or a cat I saw?" → Is palindrome: True
"Not a palindrome" → Is palindrome: False
"12321" → Is palindrome: True
"12345" → Is palindrome: False

D:\Downloads\Python_FullCourse>
```

JUSTIFICATION:

The function is `_palindrome(sentence)` checks if a given sentence is a palindrome by first cleaning the input. It removes all spaces and punctuation marks, and converts the string to lowercase to ensure that the palindrome check is case-insensitive and ignores non-alphanumeric characters. Then, it compares the cleaned sentence with its reverse to determine if it is a palindrome. This approach allows the function to accurately identify palindromic sentences while ignoring irrelevant characters, making it robust for various inputs.

QUESTION 4

Task 4: ShoppingCart Class

Prompt:

Generate a shopping cart program . the class has to include methods like `add_item(name,price)`,`remove_item(name)`,`total_cost()`. generate test cases for the shopping cart class.validate correct addition,removal and cost calculation. handle empty cart scenarios.calculate costs accurately and add and remove items correctly.

Code:

conditional logic.py > ...

```
1 import string
2
3 def is_palindrome(sentence):
4     # Remove spaces and punctuation, and convert to lowercase
5     cleaned_sentence = ''.join(
6         char for char in sentence if char.isalnum()
7     ).lower()
8
9     # Check if cleaned sentence is equal to its reverse
10    return cleaned_sentence == cleaned_sentence[::-1]
11
12
13    # ----- Test Cases -----
14
15    test_sentences = [
16        "A man, a plan, a canal Panama",
17        "No 'x' in Nixon",
18        "Was it a car or a cat I saw?",
19        "Not a palindrome",
20        "12321",
21        "12345"
22    ]
23
24    for sentence in test_sentences:
25        result = is_palindrome(sentence)
26        print(f"{sentence} → Is palindrome: {result}")
27
28    class ShoppingCart:
29        def __init__(self):
30            self.cart = {}
31
32        def add_item(self, name, price):
33            # Validate input types
34            if not isinstance(name, str) or not isinstance(price, (int, float)):
35                raise ValueError("Invalid input: name must be a string and price must be a number.")
36
37            # Validate price
38            if price < 0:
39                raise ValueError("Invalid input: price cannot be negative.")
40
41            self.cart[name] = price
```

conditional logic.py > ...

```
27 class ShoppingCart:
31     def add_item(self, name, price):
38         raise ValueError("Invalid input: price cannot be negative.")
39
40         self.cart[name] = price
41
42     def remove_item(self, name):
43         if name in self.cart:
44             del self.cart[name]
45         else:
46             raise ValueError("Item not found in cart.")
47
48     def total_cost(self):
49         return sum(self.cart.values())
50
51
52 # ----- Test Cases (Single Try Block) -----
53
54 try:
55     cart = ShoppingCart()
56
57     # Adding items
58     cart.add_item("Apple", 1.5)
59     cart.add_item("Banana", 0.75)
60     cart.add_item("Orange", 1.25)
61
62     print("Items in cart:", cart.cart)
63     print("Total cost:", cart.total_cost())
64
65     # Removing item
66     cart.remove_item("Banana")
67     print("After removing Banana:", cart.cart)
68     print("Updated total cost:", cart.total_cost())
69
70     # Invalid cases
71     cart.add_item("Grapes", -2)           # Negative price
72     cart.add_item(123, 2.5)              # Invalid name type
73     cart.remove_item("Mango")            # Item not in cart
74
75 except ValueError as e:
76     print("Error:", e)
77
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS SPELL CHECKER

Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  SPELL CHECKER

"No 'x' in Nixon" → Is palindrome: True
"Was it a car or a cat I saw?" → Is palindrome: True
"Not a palindrome" → Is palindrome: False
"12321" → Is palindrome: True
"12345" → Is palindrome: False
Items in cart: {'Apple': 1.5, 'Banana': 0.75, 'Orange': 1.25}
Total cost: 3.5
After removing Banana: {'Apple': 1.5, 'Orange': 1.25}
Updated total cost: 2.75
Error: Invalid input: price cannot be negative.

D:\Downloads\Python_FullCourse>
```

JUSTIFICATION:

The program uses a dictionary to store items and their prices in a shopping cart.

The add item() function adds items with their price.

The remove_ item() function checks if the item exists before removing it to avoid errors. The total_ cost() function calculates the total price using the sum() function .If an item is not found, the program prints an error message instead of crashing. This ensures proper error handling and clear output.

QUESTION 5

Task 5: Date Format Conversion

Prompt:

Write a Python function named `convert_date_format(date_str)` that changes a date from "YYYY-MM-DD" format to "DD-MM-YYYY" format.

For example, if the input is "2023-10-15", it should return "15-10-2023".

Test the function with different correct dates to check if it works properly.

If the date format is wrong, the function should show an error message instead of crashing.

Code:

```
conditional logic.py > ...
1 def convert_date_format(date_str):
2     try:
3         year, month, day = date_str.split('-')
4
5         # Validate format length
6         if len(year) != 4 or len(month) != 2 or len(day) != 2:
7             raise ValueError("Invalid date format: expected 'YYYY-MM-DD'.")
8
9         return f"{day}-{month}-{year}"
10
11    except ValueError as e:
12        raise ValueError(f"Invalid input: {e}")
13
14
15 # ----- Test Cases -----
16
17 test_dates = [
18     "2023-10-15",
19     "1990-01-01",
20     "2020-12-31",
21     "2023-2-5",      # Invalid format
22     "2023/10/15",    # Invalid format
23     "15-10-2023"     # Invalid format
24 ]
25
26 for date in test_dates:
27     try:
28         converted_date = convert_date_format(date)
29         print(f"Original: {date}, Converted: {converted_date}")
30     except ValueError as e:
31         print(f"Error: {e}")
32
```

Output:

```
D:\Downloads\Python_FullCourse>C:/Users/Chithra/AppData/Local/Programs/Python/Python313/python.exe "d:/Downloads/Python_FullCourse/conditional logic.py"
Original: 2023-10-15, Converted: 15-10-2023
Original: 1990-01-01, Converted: 01-01-1990
Original: 2020-12-31, Converted: 31-12-2020
Error: Invalid input: Invalid date format: expected 'YYYY-MM-DD'.
Error: Invalid input: not enough values to unpack (expected 3, got 1)
Error: Invalid input: Invalid date format: expected 'YYYY-MM-DD'.

D:\Downloads\Python_FullCourse>
```

JUSTIFICATION:

The function first checks whether the date follows the correct "YYYY-MM-DD" format. It verifies the length and position of dashes to ensure proper structure. Then it splits the date into year, month, and day and checks if they are numeric. If everything is correct, it rearranges the parts into "DD-MM-YYYY" format. If the format is wrong, it raises a clear error message instead of crashing. The test cases include both valid and invalid inputs to ensure proper validation and error handling.